

StudyPair Backend

Presentation Guide & Speaking Notes

A working script for presenting the FastAPI backend: what to demo, what to say, what to know before you walk in, and who covers what.

Repository	github.com/Sonsorilssah/studypair
Stack	FastAPI · SQLAlchemy 2.x · PostgreSQL · Alembic · Google OAuth (Authlib) · JWT
Presenters	Daniel — tutor/student booking core Sonsori — auth, admin, notifications, integration
Endpoints	31 across 4 routers: auth, tutors, students, admin

1. Before You Present — Checklist

Environment

- Start Postgres: `docker compose up -d` (from backend/)
- Activate the venv and confirm migrations are current: `alembic upgrade head`
- Start the API with reload on: `uvicorn app.main:app --reload`
- Confirm it's alive: `curl http://localhost:8000/health → {"status":"ok"}`
- Open Swagger UI at `http://localhost:8000/docs` in a browser tab — good for showing the endpoint catalog and schemas

Two things that will trip you up if you don't know about them going in

1. Swagger's “Authorize” button doesn't accept a plain token.

It renders a username/password form, because our auth dependency is declared with FastAPI's `OAuth2PasswordBearer` class (used purely to extract a Bearer header — there's no real password grant behind it). Pasting a JWT into that form does nothing useful.

SAY: If someone asks why Authorize doesn't work: “Swagger's login form assumes a password grant. We use bearer tokens from Google OAuth instead, so we drive authenticated calls with curl or Postman for this demo — the docs page is still accurate for what each endpoint expects.”

2. There is no working live Google login yet.

The redirect to Google's consent screen works and is worth showing — `GET /auth/google/login` correctly redirects with the right `client_id`, `state`, and `nonce` parameters. But finishing a real login needs a registered Google Cloud OAuth app (client ID + secret in `.env`), which hasn't been set up yet.

SAY: “This redirect is real — it's hitting Google's actual OAuth endpoint. Completing the login needs a registered OAuth app on Google's side, which is an infra step, not a code gap.”

Pre-mint demo accounts and tokens (do this before you present, not live)

There's no working login yet, so “logging in” for the demo means creating a user row the same shape Google's callback would produce, then minting a real JWT for it directly — exactly what the actual login code does after it gets a profile back from Google. Run this once beforehand and keep the three tokens somewhere you can paste from quickly (a notes file — not scrollback).

```
cd backend && source venv/Scripts/activate
python -c "
from app.core.database import SessionLocal
from app.core.security import create_access_token
from app.models.user import User

db = SessionLocal()
tutor = User(google_id='demo-tutor', email='demo.tutor@example.com', full_name='Demo Tutor')
student = User(google_id='demo-student', email='demo.student@example.com', full_name='Demo Student')
admin = User(google_id='demo-admin', email='demo.admin@example.com', full_name='Demo Admin', is_admin=True)
db.add_all([tutor, student, admin]); db.commit()
for u in (tutor, student, admin): db.refresh(u)
```

```
print('TUTOR_TOKEN=' + create_access_token(tutor.id))
print('STUDENT_TOKEN=' + create_access_token(student.id))
print('ADMIN_TOKEN=' + create_access_token(admin.id))
db.close()
"
```

Export each printed line as a shell variable (`export TUTOR_TOKEN=...`) in the terminal you'll present from, so every curl command below just works by reference.

Final checks

- Do one full dry run of your demo sequence today, not last week — don't present a flow you haven't run recently
- If you've run the demo before, wipe the throwaway rows first (or restart the Postgres container fresh) so course/slot names don't collide with duplicate-check errors mid-presentation
- Know your fallback: if live calls misbehave, you have this document's exact expected responses to fall back on verbally

2. Opening — System Overview

Whoever opens the presentation, use this to frame the whole system before splitting into parts.

SAY: StudyPair is a peer-tutoring platform for university students. The backend is FastAPI with SQLAlchemy and PostgreSQL, migrations managed through Alembic. Auth is Google-only — no passwords for us to store or manage, which matches how students already sign into everything else at school and keeps the platform free with no accounts to reset.

SAY: A user can be both a tutor and a student at the same time. There's no role field on the user — it's purely behavioral: if you've listed a course you teach, the system treats you as a tutor wherever that's relevant.

SAY: We split ownership along the two halves of the product. Daniel owns the booking core — tutors listing courses and availability, students browsing and requesting slots, the accept/reject flow. I own everything around that: signing in, the admin visibility tools, notifications, and after our pieces merged, an integration pass across the whole thing.

Two design points worth calling out early — they come up in Q&A:

- **No double-booking, even under a race.** A slot locks the instant a request is made, not when the tutor accepts — with a row-level lock so two students hitting “request” in the same instant can't both win.
- **No cron jobs.** Match requests (48h) and course applications (2 days) expire lazily — checked and swept the moment something reads them, not on a schedule. Simpler to reason about, nothing to keep alive.

3. Daniel's Part — Tutor & Student Booking Core

Owens: routers/tutors.py, the browsing/booking half of routers/students.py, services/expiry.py, services/slots.py, services/course_names.py, schemas/{common, tutors, students}.py.

Demo sequence (build the whole booking lifecycle live, ~2 minutes)

Step 1 — Tutor lists a course

```
curl -s -X POST http://localhost:8000/tutors/me/courses \
-H "Authorization: Bearer $TUTOR_TOKEN" -H "Content-Type: application/json" \
-d '{"course_name": "Calculus I"}'
```

→ 201, returns the course plus a fulfilled-applications count (0 here).

SAY: Try posting it again with different casing/spacing — “calculus i” — to show the 409 duplicate check.

Step 2 — Tutor adds a weekly slot

```
curl -s -X POST http://localhost:8000/tutors/me/availability \
-H "Authorization: Bearer $TUTOR_TOKEN" -H "Content-Type: application/json" \
-d '{"day_of_week": "Mon", "start_time": "10:00:00", "end_time": "11:00:00"}'
```

→ 201. Recurring weekly — not tied to a specific calendar date.

Step 3 — Student browses

```
curl -s "http://localhost:8000/students/tutors?course=calc" \
-H "Authorization: Bearer $STUDENT_TOKEN"
```

→ 200, the tutor shows up with the course and the open slot.

Step 4 — Student requests the slot

```
curl -s -X POST http://localhost:8000/students/requests \
-H "Authorization: Bearer $STUDENT_TOKEN" -H "Content-Type: application/json" \
-d '{"course_id": "<course_id_from_step_1>", "slot_id": "<slot_id_from_step_2>"}'
```

SAY: The slot locks right now, not when the tutor accepts — that's what stops two students racing for the same hour. It's enforced with a row-level lock at the database layer, so it holds even under real concurrency.

Step 5 — Tutor sees it and accepts

```
curl -s http://localhost:8000/tutors/me/requests -H "Authorization: Bearer $TUTOR_TOKEN"

curl -s -X POST http://localhost:8000/tutors/me/requests/<request_id>/accept \
-H "Authorization: Bearer $TUTOR_TOKEN"
```

SAY: Accepted is permanent in v1 — there's no session-completion or no-show tracking. That was a deliberate scope decision, not an oversight, to keep the first version shippable.

Step 6 — The nice one: an application getting fulfilled automatically

```
curl -s -X POST http://localhost:8000/students/course-applications \
-H "Authorization: Bearer $STUDENT_TOKEN" -H "Content-Type: application/json" \
-d '{"course_name": "Organic Chemistry"}'

curl -s -X POST http://localhost:8000/tutors/me/courses \
-H "Authorization: Bearer $TUTOR_TOKEN" -H "Content-Type: application/json" \
-d '{"course_name": "Organic Chemistry"}'
```

SAY: A student can ask for a course nobody teaches yet. When a tutor later lists a matching course, the application is auto-fulfilled and the student gets a notification — but no booking is auto-created, they still browse and request normally. That's a deliberate product call: the system tells you demand was met, it doesn't book on your behalf.

Talking points to weave in

- Course names are free text, matched case- and whitespace-insensitively — one rule shared by both the “do I already teach this” check and the “does this fulfil an application” check, so they can't drift apart
- Lazy expiry: 48 hours for an unanswered match request, 2 days for an unmet course application — checked and swept on every read, no background job
- Segue to Sonori's part: “In my PR I flagged an open question — what happens if you delete a course or slot that still has match requests against it? We resolved that together once the pieces merged.”

4. Sorsori's Part — Auth, Admin, Notifications & Integration

Owens: core/security.py, routers/auth.py, routers/admin.py, services/notifications.py, schemas/{auth, notification}.py, the self-service half of routers/students.py (view/cancel/withdraw own requests and applications, mark notifications read), plus the is_admin migration.

Demo sequence

Step 1 — Prove auth is real: show the redirect, then the gate

```
curl -sI http://localhost:8000/auth/google/login | grep -i location

curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/auth/me

curl -s http://localhost:8000/auth/me -H "Authorization: Bearer $STUDENT_TOKEN"
```

→ Redirect to accounts.google.com, then 401 with no token, then 200 with one.

Step 2 — Complete a profile

```
curl -s -X POST http://localhost:8000/auth/complete-profile \
-H "Authorization: Bearer $STUDENT_TOKEN" -H "Content-Type: application/json" \
-d '{"phone_number": "024-555-0142"}'
```

→ profile_completed flips from false to true.

Step 3 — Admin gate

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/admin/users \
-H "Authorization: Bearer $STUDENT_TOKEN"

curl -s http://localhost:8000/admin/users -H "Authorization: Bearer $ADMIN_TOKEN"

curl -s "http://localhost:8000/admin/course-applications?status=open" \
-H "Authorization: Bearer $ADMIN_TOKEN"
```

SAY: Admin is a single boolean on the user — is_admin, set directly in the database for now. No roles table, no permission matrix. That's deliberate: it matches DECISIONS.md, the admin tooling is for the dev team only, not a public feature, so it doesn't need more than that yet.

Step 4 — What I added on top of Daniel's dashboard

After the first merge, a student could book and apply, but never see their own history, and couldn't back out of anything. I closed that gap:

```
curl -s http://localhost:8000/students/me/requests -H "Authorization: Bearer $STUDENT_TOKEN"

curl -s -X POST http://localhost:8000/students/me/requests/<request_id>/cancel \
-H "Authorization: Bearer $STUDENT_TOKEN"

curl -s -X PATCH http://localhost:8000/students/me/notifications/<id>/read \
-H "Authorization: Bearer $STUDENT_TOKEN"
```

SAY: Cancelling only works from pending — accepted is still permanent, matching the same rule Daniel's accept/reject code already enforces. I didn't invent a new rule, I extended the existing one to a case it hadn't covered yet.

Step 5 — Tell the engineering-process story (this is worth saying out loud)

SAY: “When Daniel's dashboard PR merged, his own description flagged that it had never actually run against a live database — Docker wasn't available in his environment, so it was only checked by import and schema generation. I wrote a full integration script and ran every endpoint against real Postgres. It found a real bug: deleting a course that only had old, resolved match-request history against it crashed with a database error instead of just working. I fixed the underlying rule — only an active request should ever block a delete, not permanently-resolved history — and re-verified it.”

SAY: “I found a second one myself, before it ever touched real data. Adding the new cancelled/withdrawn statuses, I assumed the database stored them lowercase to match our JSON. I checked empirically against the live database first instead of trusting that assumption — the existing values were actually uppercase. Caught it in a throwaway test, not in front of a user.”

These two stories are the strongest material in the whole presentation — they show verification discipline, not just “we wrote code.” Don't rush past them.

5. “Easy to Test” Shortlist

If time is short, or something goes wrong mid-demo, fall back to this list — ordered by how little can go wrong.

Tier 1 — zero setup, cannot fail

- GET /health, /auth/health, /tutors/health, /students/health, /admin/health — all return {"status": "ok"}
- GET /auth/google/login — 302 redirect to Google, no account needed
- GET /auth/me with no header — clean 401

Tier 2 — needs one pre-minted token

- GET /auth/me with a token — 200 and a real profile
- GET /students/tutors — 200 even with nothing to show yet
- GET /admin/users as a non-admin — clean 403

Tier 3 — the full booking lifecycle (rehearse this one specifically)

Add course → add slot → browse → request → accept. Five calls, one straight line, shows the whole product in under two minutes. This is the sequence in Section 3 — walk through it once today before presenting so the UUIDs-in-the-URL friction doesn't surprise you live.

6. Anticipated Questions

O: Why is there no automated test suite yet?

A: “There isn't one yet — it's the top priority right after this. Everything shown today was verified by hand against a real database this week. That's exactly why an automated suite and CI are next: manual verification doesn't scale, and we already found two real bugs doing it by hand.”

O: What actually stops two students booking the same slot at once?

A: “The slot locks the instant a request is made, not when it's accepted, and that lock is taken with a database row lock — so even two requests arriving in the same instant serialize correctly instead of racing.”

O: What happens if a tutor deletes a course a student already booked?

A: “It can't happen silently. A pending or accepted request blocks the delete entirely. Only resolved history — rejected or expired requests — gets cleared out of the way, and even then only as part of that same delete.”

O: Why Google-only login instead of email/password?

A: “It's a free platform for university students — no payments, and Google matches how they already sign into everything else at school. It also means we never store or manage a password.”

O: Can one person be both a tutor and a student?

A: “Yes — there's no role field at all. It's purely behavioral: having listed courses is what makes you a tutor wherever the system needs to know.”

O: What's left before this could actually go live?

A: “A real registered Google OAuth app, the JWT secret changed from its placeholder, an automated test suite and CI, a deployment target, and a couple of small feature gaps we're tracking — nothing structural, just execution.”

Appendix — Full Endpoint Reference

Leave-behind reference. Not meant to be read line by line during the presentation.

auth — Sponsori

Method	Path	Auth
GET	/auth/health	none
GET	/auth/google/login	none
GET	/auth/google/callback	none (Google redirect)
POST	/auth/complete-profile	bearer
GET	/auth/me	bearer

tutors — Daniel

Method	Path	Auth
GET	/tutors/health	none
GET	/tutors/me/requests	bearer
POST	/tutors/me/requests/{request_id}/accept	bearer
POST	/tutors/me/requests/{request_id}/reject	bearer
POST	/tutors/me/courses	bearer
DELETE	/tutors/me/courses/{course_id}	bearer
POST	/tutors/me/availability	bearer
DELETE	/tutors/me/availability/{slot_id}	bearer

students — Daniel (browsing/booking) + Sponsori (self-service, marked *)

Method	Path	Auth
GET	/students/health	none
GET	/students/tutors	bearer
POST	/students/requests	bearer
POST	/students/course-applications	bearer
GET	/students/me/notifications	bearer
GET	/students/me/requests *	bearer
POST	/students/me/requests/{request_id}/cancel *	bearer
GET	/students/me/course-applications *	bearer
POST	/students/me/course-applications/{application_id}/withdraw *	bearer

Method	Path	Auth
PATCH	/students/me/notifications/{notification_id}/read *	bearer

admin — Sonsori

Method	Path	Auth
GET	/admin/health	none
GET	/admin/course-applications	bearer + is_admin
GET	/admin/users	bearer + is_admin

Generated for the StudyPair team — verified against a live PostgreSQL database, not just import-checked.